

Concurrency

1 Producer/consumer probleem

Geef een mogelijke implementatie voor het producer/consumer probleem met semaforen, in de veronderstelling dat de buffer waarin de data wordt weggeschreven nu eindig is; met andere woorden, als de producer een item wilt wegschrijven en de buffer is vol, moet deze wachten tot er terug een plek vrij is.

Solution:

<pre>producer: repeat produce; wait(e); wait(s); append; signal(s); signal(n); forever</pre>	<pre>consumer: repeat wait(n); wait(s); take; signal(s); signal(e); consume; forever</pre>
--	--

2 Concurrency

Beschouw het volgende stuk code:

```
const int n = 50;
int tally;
void total() {
    int count;
    for (count = 1; count <= n; count++) {
        tally=tally+1;
    }
}

void main() {
    tally = 0;
    parbegin(total(),total()); // parallel uitvoeren
    cout << tally << endl;
}
```

1. Wat is de minimale en maximale waarde die de variabele *tally* kan aannemen, bij het einde van het programma?

Solution: De instructie `tally=tally+1` wordt niet atomair uitgevoerd en bestaat dus uit `read`, `increment` en `write`.

- Maximaal 100

- Minimaal 2
 P_1 : read(0)
 P_2 : read(0), 49 iteraties, write(49)
 P_1 : increment, write(1)
 P_2 : read(1)
 P_1 : read(1), 49 iteraties, write(50)
 P_2 : increment, write(2)

2. Stel dat er N parallele instanties van total worden uitgevoerd, wat verandert er aan deze grenzen?

Solution: De instructie `tally=tally+1` wordt niet atomair uitgevoerd en bestaat dus uit `read`, `increment` en `write`.

- Maximaal $50N$.
- Minimaal 2
 P_1 : read(0)
 P_2 : read(0), 49 iteraties, write(49)
 P_1 : increment, write(1)
 P_2 : read(1)
 P_1 : read(1), 49 iteraties, write(50)
 $P_{3...N}$: read(0), 50 iteraties, write(50)
 P_2 : increment, write(2)

3 Paper review

Papers reviewen behoort tot de talrijke activiteiten van onderzoekers. Het doel hiervan bestaat erin om een paper “na te kijken”, onder andere op correctheid, alvorens deze kan gepubliceerd worden in een tijdschrift, of voorgesteld worden op een conferentie. Onderstaande *comment* is in 1965 ingestuurd naar het vrij hoog aangeschreven tijdschrift Communications of the ACM. Review deze comment, in Figuur 2, op correctheid. (Tip: zoek een uitvoervolgorde, waarbij twee processen tegelijkertijd toegang verkrijgen tot de kritische sectie)

Solution: Het algoritme komt overeen met:

```
0: b[i]:=0;
1: while (k != i) {
2:   while (!b[j]) {};
3:   k:=i;
4: }
5: <critical section>
6: b[i]:=1;
```

Er kunnen twee processen tegelijkertijd in de kritische sectie komen, wanneer er een

processswitch optreedt net voor $k:=i$.

P_0 : 0,1,2

P_1 : 0,1,5: <critical section>

P_0 : 3,1,5: <critical section>

✓ Comments on a Problem in Concurrent Programming Control

Dear Editor:

I would like to comment on Mr. Dijkstra's solution [Solution of a problem in concurrent programming control. *Comm ACM* 8 (Sept. 1965), 569] to a messy problem that is hardly academic. We are using it now on a multiple computer complex.

When there are only two computers, the algorithm may be simplified to the following:

Boolean array $b(0; 1)$ **integer** k, i, j ,

comment This is the program for computer i , which may be either 0 or 1, computer $j \neq i$ is the other one, 1 or 0;

$C0$: $b(i) := \text{false}$;

$C1$: **if** $k \neq i$ **then begin**

$C2$: **if not** $b(j)$ **then go to** $C2$;

else $k := i$; **go to** $C1$ **end**;

else critical section;

$b(i) := \text{true}$;

remainder of program;

go to $C0$;

end

Mr. Dijkstra has come up with a clever solution to a really practical problem.

HARRIS HYMAN

Munitype

New York, New York

Figuur 2: Comment submitted to Communications of the ACM.

4 Binaire semaforen

```
void semWait(semaphore s) {
    semWaitB(mutex);
    s--;
    if (s < 0) {
        semSignalB(mutex);
        semWaitB(delay);
    } else
        semSignalB(mutex);
}

void semSignal(semaphore s) {
    semWaitB(mutex);
    s++;
    if (s <= 0)
        semSignalB(delay);
    semSignalB(mutex);
}
```

Deze code zou een mogelijke implementatie kunnen voorstellen, die *counting semaphores* realiseert, aan de hand van binaire semaforen. Helaas kan deze code nog een probleem opleveren. Zoek dit probleem, en pas de code aan zodat je een correcte oplossing verkrijgt.

Solution: Stel, twee processen starten `semWait`. Beiden worden onderbroken net voor `semWaitB(delay)`. Stel bovendien dat twee andere processen `semSignal` starten en deze functie beiden volledig doorlopen. Nu is `semSignalB(delay)` twee maal uitgevoerd. Dit zijn echter binaire semaforen, dus wanneer de twee eerste processen verderlopen en `semWaitB(delay)` uitvoeren, zal slechts een van hen kunnen verdergaan. De andere moet onterecht blijven wachten.

```
void semWait(semaphore s) {
    semWaitB(mutex);
    s--;
    if (s < 0) {
        semSignalB(mutex);
        semWaitB(delay);
    }
    semSignalB(mutex);
}

void semSignal(semaphore s) {
    semWaitB(mutex);
    s++;
    if (s <= 0)
        semSignalB(delay);
    else
        semSignalB(mutex);
}
```

5 Mutual exclusion

Gegeven de onderstaande code, beantwoord de volgende vragen.

- Is mutual exclusion gegarandeerd voor de kritische sectie? Leg uit.
- Is het mogelijk om in een deadlock terecht te komen? Leg uit.
- Is starvation mogelijk? Leg uit.

```
1 globale variabelen:
2 int turn = -1;
3 bool busy = false;
4
5 thread_i:
6     while(turn != i) {
7         while(busy) {
8             turn = i;
9         }
10        busy = true;
11    }
12 }
13
14 <critical section>
15
16 busy = false;
```

Solution:

- In de gegeven code eist thread i eerst zijn beurt op, dan controleert hij of er niemand anders al bezig is, indien niet zegt hij dat hij zelf bezig is om vervolgens te controleren of er ondertussen niemand anders de beurt heeft opgeëist. Deze volgorde zorgt ervoor dat een thread de outer loop slechts kan verlaten als het zijn beurt is en als `busy == true`. Daardoor kan er niemand meer uit de inner loop geraken als er iemand uit de outer is geraakt. Voorts, als we veronderstellen dat er verschillende threads uit de inner loop zijn geraakt maar nog uit de outer loop moeten geraken, kan er maar 1 van deze threads uit de outer geraken: nl. diegene die het laatste `turn = i` heeft uitgevoerd, al de overigen gaan terug naar lijn 7 springen omdat het niet hun beurt is en komen dus opnieuw terecht in de inner loop.
- Het is mogelijk om in een deadlock terecht te komen. Stel bv dat er van thread 1 naar thread 2 gewisseld wordt op het moment dat thread 1 tussen lijn 10 en 11 zit (maw net `busy = true` heeft uitgevoerd) en thread 2 zich vlak voor lijn 8 bevindt. In dat geval zal thread 2 `turn = 2` uitvoeren en vervolgens zal deze thread controleren of `busy` true is. Dit laatste is het geval is en dus zal deze thread in de loop blijven hangen. Als we dan terug wisselen naar thread 1, dan zal deze thread controleren op `turn != 1`, wat ook het geval is aangezien thread 2 de waarde van `turn` op 2 heeft gezet. Thread 1 zal dus niet uit de outer loop springen, waardoor

hij opnieuw in de inner loop terecht komt en daarin zal blijven hangen aangezien `busy` `true` is. Beide threads hangen nu in de inner loop en dit zal zo blijven tenzij er een andere thread de waarde van `busy` op `false` zet.

- Starvation kan ook voorkomen. Stel dat er van thread 1 naar thread 2 wordt gewisseld als thread 1 net lijn 10 wilt gaan uitvoeren (net na de inner loop dus). Er is niets dat de andere threads nu tegenhoudt om de waarde van `turn` aan te passen en thread 1 voorbij te steken, ook al zijn ze later toegekomen dan thread 1.

6 Model van een pretpark

Beschouw een attractie in een pretpark bestaande uit een rondrit in een 1-persoonswagentje door een spookhuis. In totaal beschikt de attractie over n wagentjes op een vast spoor (de wagentjes kunnen elkaar niet inhalen) en om energie te besparen, rijden enkel de volle wagentjes rond; de overigen staan in het station te wachten op passagiers. De m bezoekers van het pretpark lopen een tijd rond in het pretpark, komen aan bij het spookhuis, wachten eventueel op een wagentje in een FIFO queue, stappen in het eerst beschikbare wagentje in, genieten van de attractie, stappen uit en vervolledigen dan hun bezoek aan het park.

Als we de bezoekers en de wagentjes beide modelleren als processen die zich moeten synchroniseren (gebruik makend van semaforen), moet er zeker voorkomen worden dat

- Een wagentje kan vertrekken terwijl het nog niet vol is.
- Een wagentje kan vertrekken terwijl een passagier in of uit dat wagentje stapt.
- Passagiers te vroeg uitstappen, i.e. ze mogen het wagentje niet verlaten tijdens de rit.

Gegeven de globale variabelen:

```
cars_avail = semaphore initialized on 0;
passenger_safe[] = array of n semaphores initialized on 0;
ride_done[] = array of n semaphores initialized on 0;
```

Volledige code voor de wagentjes:

```
car_j:
while (turned_on) {
    signal(cars_avail)
    <pick up human i>
    wait(passenger_safe[j])
    <drive around with human i>
    signal(ride_done[j])
    <drop off human i>
    wait(passengers_safe[j])
}
```

Gedeeltelijke code voor de passagiers:

```

human_i:
...
<wander around in park>
...
<get into car j>
...
<get scared>
...
<get out of car j>
...
<complete visit to park>
...

```

- Vervolledig de code van de bezoekers. Maak hierbij enkel gebruik van de gekende semafooroperaties.

Solution:

```

human_i:
    <wander around in park>
    wait(cars_avail)
    <get into car j>
    signal(passenger_safe[j])
    <get scared>
    wait(ride_done[j])
    <get out of car j>
    signal(passenger_safe[j])
    <complete visit to park>

```

- Wat moet er veranderd worden indien de wagentjes k personen kunnen vervoeren? Geef de aangepaste code voor zowel de wagentjes als de bezoekers. We gaan er hierbij van uit dat passagiers onderling niet communiceren, en niet kunnen achterhalen hoeveel passagiers er reeds in het wagentje aanwezig zijn. (2pt)

Solution:

```

cars_avail = semaphore initialized on 0;
passenger_safe[] = array of n semaphores initialized on 0;
ride_done[] = array of n semaphores initialized on 0;

car_j:
while (turned_on) {
    for (int p = 0; p < k; p++) {
        signal(cars_avail)

```

```

    }
    <pick up human i..i+k>
    for (int p = 0; p < k; p++) {
        wait(passenger_safe[j])
    }
    <drive around with human i..i+k>
    for (int p = 0; p < k; p++) {
        signal(ride_done[j])
    }
    <drop off human i..i+k>
    for (int p = 0; p < k; p++) {
        wait(passenger_safe[j])
    }
}

human_i:
    <wander around in park>
    wait(cars_avail)
    <get into car j>
    signal(passenger_safe[j])
    <get scared>
    wait(ride_done[j])
    <get out of car j>
    signal(passenger_safe[j])
    <complete visit to park>

```

- Zijn er randvoorwaarden op k en m ? Leg uit. (1pt)

Solution: Indien $k > m$ zouden de paar passagiers die een ritje willen doen oneindig lang wachten tot de wagentjes vol zijn.